In [ ]:
```
df = pd.read_csv('/content/MARS_ATS_1H_CONTINUOUS.csv')

df.reset_index(drop=True, inplace=True)

print(df.shape)
```

```
(2076, 7)
```

In [ ]:
```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/MARS_ATS_1H_CONTINUOUS.csv')

print(df.shape)
df.head()
```

```
(2076, 7)
```

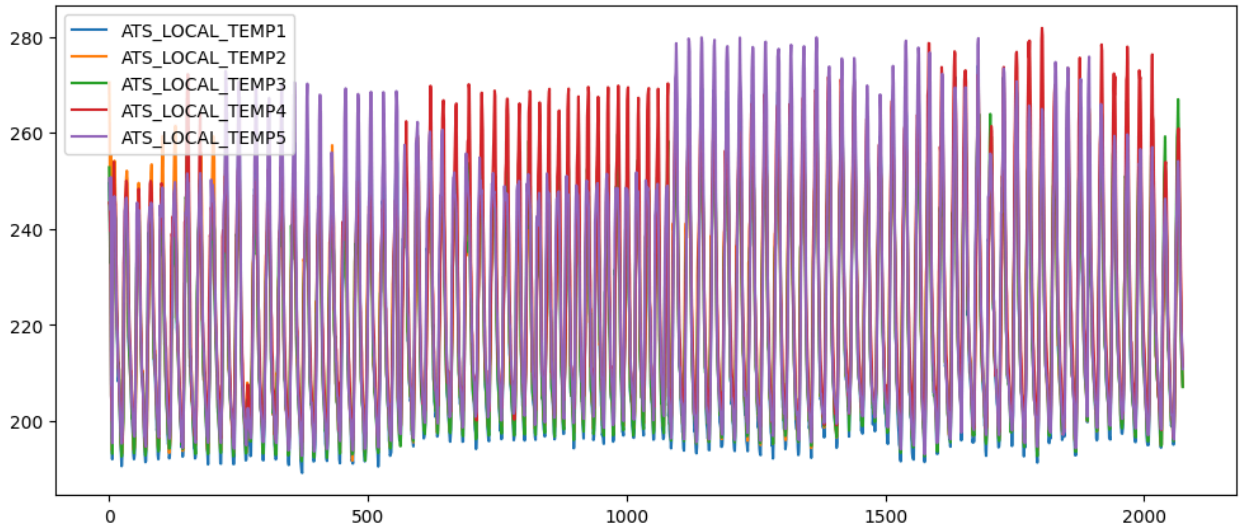
Out[ ]:

	SCLK	ATS_LOCAL_TEMP1	ATS_LOCAL_TEMP2	ATS_LOCAL_TEMP3	ATS_LOCA
0	7720 days 09:33:50	249.525532	270.451660	252.801554	2
1	7721 days 10:33:50	244.340084	260.965649	243.736995	2
2	7728 days 09:33:50	235.394630	234.195248	232.923582	2
3	7729 days 13:33:50	244.439728	255.418648	242.270108	2
4	7734 days 01:33:50	204.903284	205.313609	205.072629	2

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

In [ ]:
```
df[['ATS_LOCAL_TEMP1',
    'ATS_LOCAL_TEMP2',
    'ATS_LOCAL_TEMP3',
    'ATS_LOCAL_TEMP4',
    'ATS_LOCAL_TEMP5']].plot(figsize=(12,5))
```

Out[ ]: <Axes: >



In [ ]:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Load dataset
df = pd.read_csv("MARS_ATS_1H_CONTINUOUS.csv")

# Select temperature columns
cols = ['ATS_LOCAL_TEMP1','ATS_LOCAL_TEMP2','ATS_LOCAL_TEMP3',
        'ATS_LOCAL_TEMP4','ATS_LOCAL_TEMP5']

data = df[cols].dropna()

# Scaling
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

# Create sequences
def create_sequences(data, time_steps=24):
    X, y = [], []
    for i in range(len(data) - time_steps):
        X.append(data[i:i+time_steps])
        y.append(data[i+time_steps])
    return np.array(X), np.array(y)

X, y = create_sequences(scaled_data, 24)

# Train test split
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

In [ ]:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
```

```
model_lstm = Sequential([
    LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.sha
    Dropout(0.2),
    LSTM(32),
    Dense(5)   # 5 output temperatures
])

model_lstm.compile(optimizer='adam', loss='mse')

history_lstm = model_lstm.fit(X_train, y_train,
                              epochs=30,
                              batch_size=32,
                              validation_data=(X_test, y_test))
```

```
Epoch 1/30
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(**kwargs)
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - loss: 0.1087 - val_loss: 0.0456
Epoch 2/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0299 - val_loss: 0.0092
Epoch 3/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0113 - val_loss: 0.0087
Epoch 4/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0106 - val_loss: 0.0080
Epoch 5/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0090 - val_loss: 0.0081
Epoch 6/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0074 - val_loss: 0.0079
Epoch 7/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0081 - val_loss: 0.0078
Epoch 8/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0074 - val_loss: 0.0088
Epoch 9/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0068 - val_loss: 0.0070
Epoch 10/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0064 - val_loss: 0.0071
Epoch 11/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0057 - val_loss: 0.0067
Epoch 12/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0058 - val_loss: 0.0062
Epoch 13/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0058 - val_loss: 0.0063
Epoch 14/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0055 - val_loss: 0.0059
Epoch 15/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0052 - val_loss: 0.0061
Epoch 16/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0049 - val_loss: 0.0058
Epoch 17/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0048 - val_loss: 0.0058
Epoch 18/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0043 - val_loss: 0.0052
Epoch 19/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 0.0046 - val_loss: 0.0053
Epoch 20/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 0.0046 - val_loss: 0.0057
Epoch 21/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0045 - val_loss: 0.0051
Epoch 22/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0041 - val_loss: 0.0054
Epoch 23/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0048 - val_loss: 0.0058
Epoch 24/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0038 - val_loss: 0.0050
Epoch 25/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0045 - val_loss: 0.0045
Epoch 26/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0039 - val_loss: 0.0051
Epoch 27/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0034 - val_loss: 0.0055
Epoch 28/30
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0041 - val_loss: 0.0063
Epoch 29/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0040 - val_loss: 0.0047
Epoch 30/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0039 - val_loss: 0.0047
```

In [ ]:

```
from tensorflow.keras.layers import GRU

model_gru = Sequential([
    GRU(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shap
    Dropout(0.2),
    GRU(32),
    Dense(5)
])

model_gru.compile(optimizer='adam', loss='mse')

history_gru = model_gru.fit(X_train, y_train,
                            epochs=30,
                            batch_size=32,
                            validation_data=(X_test, y_test))
```

```
Epoch 1/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0834 - val_loss: 0.0141
Epoch 2/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0116 - val_loss: 0.0080
Epoch 3/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0083 - val_loss: 0.0073
Epoch 4/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0075 - val_loss: 0.0068
Epoch 5/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0077 - val_loss: 0.0065
Epoch 6/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0069 - val_loss: 0.0060
Epoch 7/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0062 - val_loss: 0.0054
Epoch 8/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0058 - val_loss: 0.0055
Epoch 9/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0054 - val_loss: 0.0056
Epoch 10/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0058 - val_loss: 0.0052
Epoch 11/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0052 - val_loss: 0.0050
Epoch 12/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0049 - val_loss: 0.0048
Epoch 13/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0045 - val_loss: 0.0046
Epoch 14/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0051 - val_loss: 0.0046
Epoch 15/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0046 - val_loss: 0.0046
Epoch 16/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0055 - val_loss: 0.0047
Epoch 17/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0045 - val_loss: 0.0045
Epoch 18/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0048 - val_loss: 0.0046
Epoch 19/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0038 - val_loss: 0.0043
Epoch 20/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0044 - val_loss: 0.0042
Epoch 21/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0041 - val_loss: 0.0045
Epoch 22/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0043 - val_loss: 0.0042
Epoch 23/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0047 - val_loss: 0.0042
Epoch 24/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0041 - val_loss: 0.0045
Epoch 25/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0042 - val_loss: 0.0043
Epoch 26/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0037 - val_loss: 0.0040
Epoch 27/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0042 - val_loss: 0.0043
```

```
Epoch 28/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0038 - val_loss: 0.0047
Epoch 29/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0039 - val_loss: 0.0041
Epoch 30/30
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0038 - val_loss: 0.0042
```

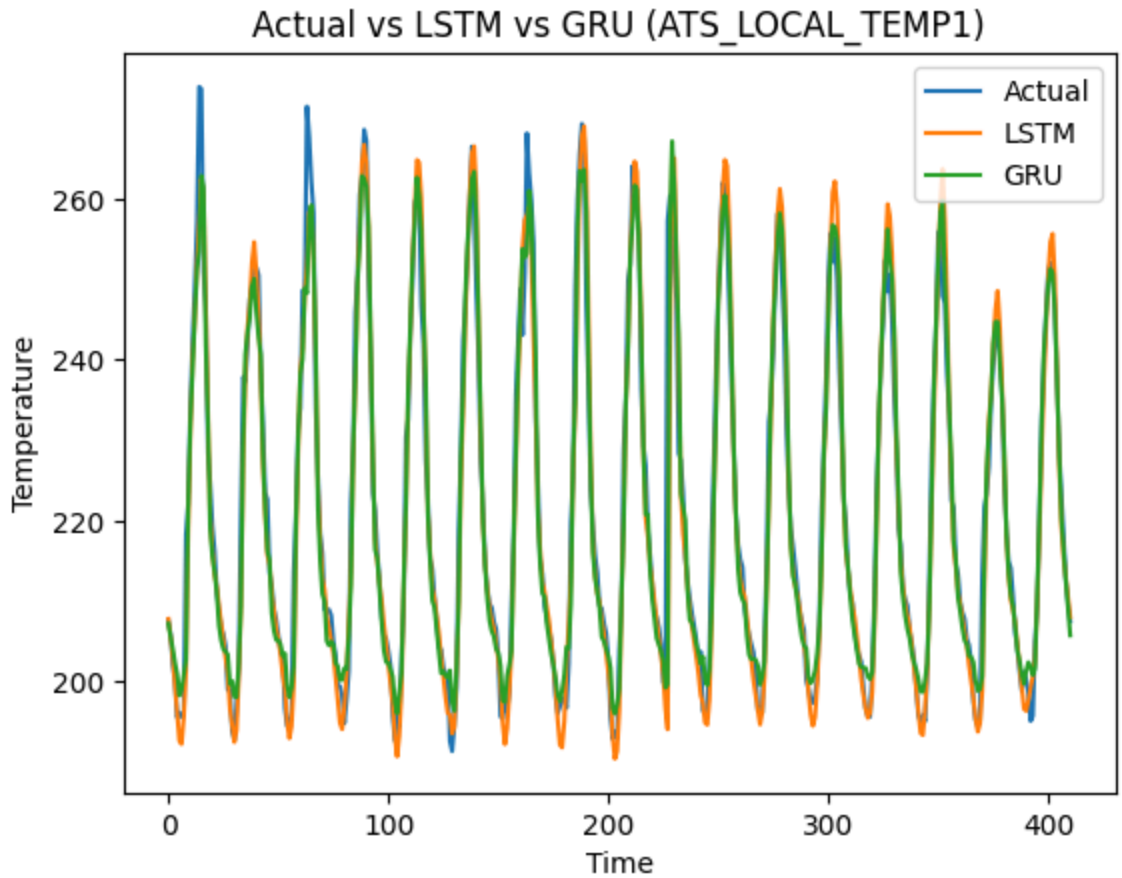
In [ ]:

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

y_pred_lstm = model_lstm.predict(X_test)
y_pred_gru = model_gru.predict(X_test)

print("LSTM RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lstm)))
print("GRU RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_gru)))

print("LSTM MAE:", mean_absolute_error(y_test, y_pred_lstm))
print("GRU MAE:", mean_absolute_error(y_test, y_pred_gru))

print("LSTM R2:", r2_score(y_test, y_pred_lstm))
print("GRU R2:", r2_score(y_test, y_pred_gru))
```

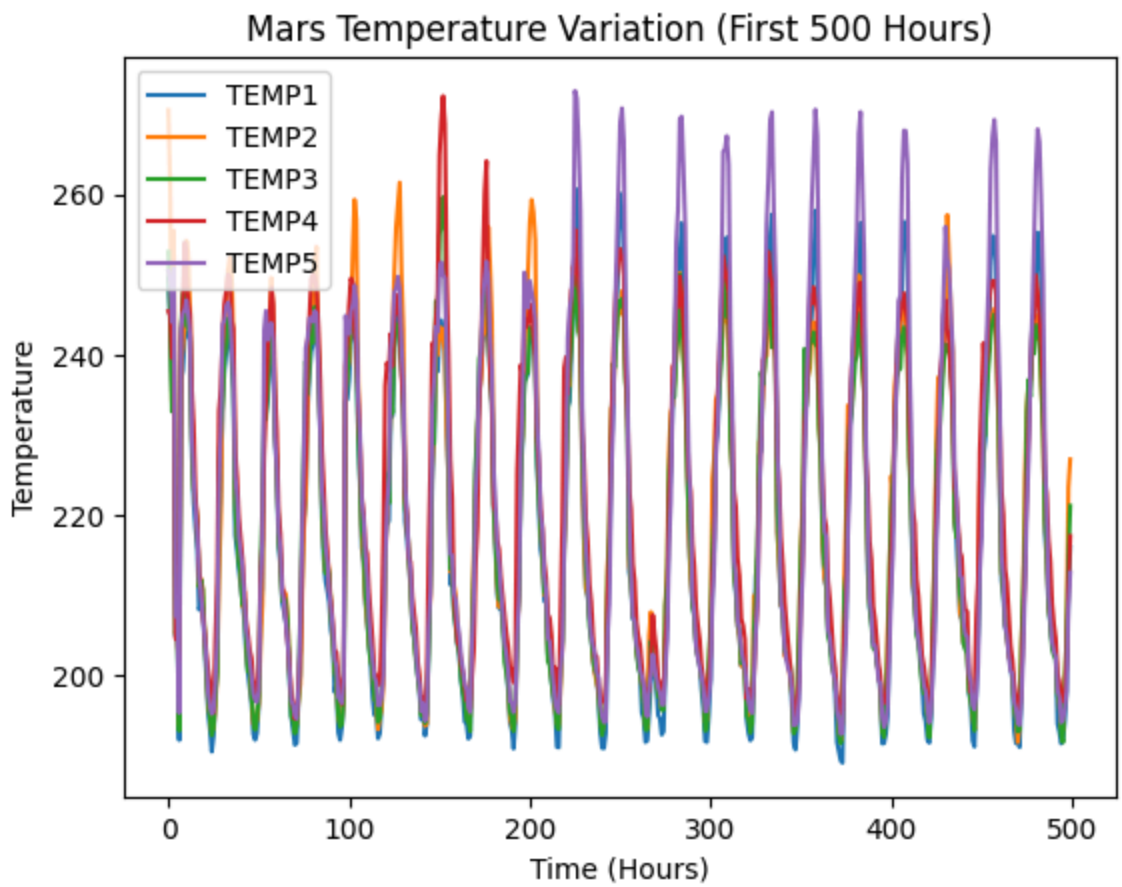
```
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step
LSTM RMSE: 0.06875548681833292
GRU RMSE: 0.06450729530821472
LSTM MAE: 0.041930728451008384
GRU MAE: 0.0431560137158462
LSTM R2: 0.9338568059673692
GRU R2: 0.9419069738770875
```

In [ ]:

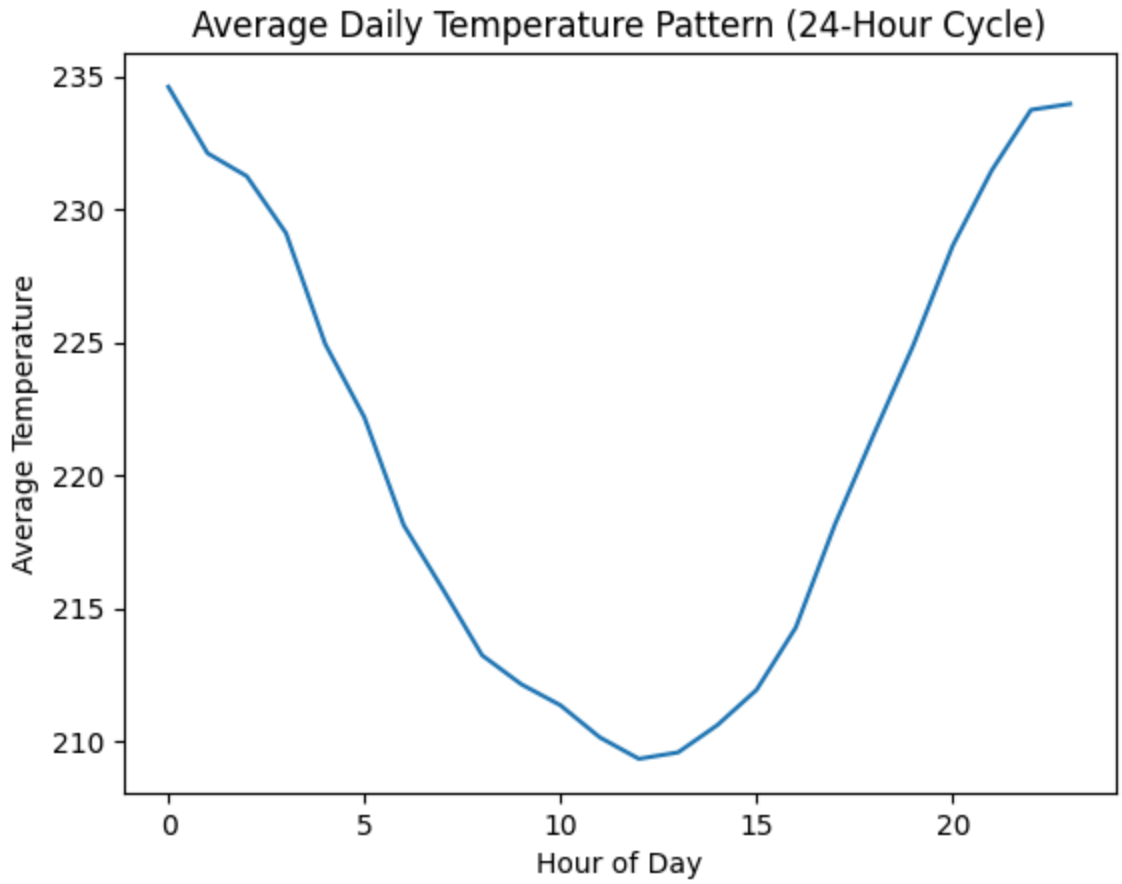
```
import matplotlib.pyplot as plt
import numpy as np

# Choose one temperature for visualization (e.g., ATS_LOCAL_TEMP1)
temp_index = 0  # 0 = TEMP1

# Inverse scaling
y_test_inv = scaler.inverse_transform(y_test)
lstm_pred_inv = scaler.inverse_transform(y_pred_lstm)
gru_pred_inv = scaler.inverse_transform(y_pred_gru)

plt.figure()
plt.plot(y_test_inv[:, temp_index])
plt.plot(lstm_pred_inv[:, temp_index])
plt.plot(gru_pred_inv[:, temp_index])
plt.title("Actual vs LSTM vs GRU (ATS_LOCAL_TEMP1)")
plt.xlabel("Time")
plt.ylabel("Temperature")
plt.legend(["Actual", "LSTM", "GRU"])
plt.show()
```



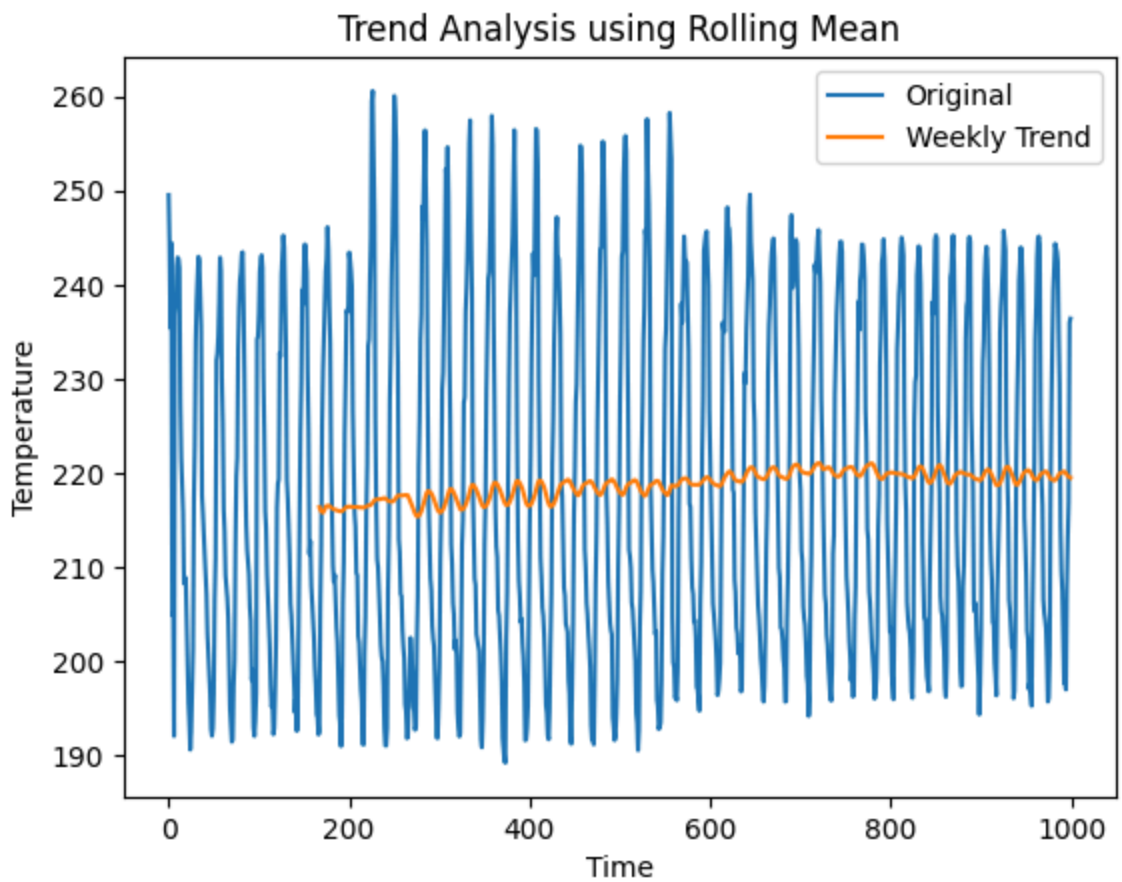
In [ ]:
```
plt.figure()
for i in range(5):
    plt.plot(data.iloc[:500, i])

plt.title("Mars Temperature Variation (First 500 Hours)")
plt.xlabel("Time (Hours)")
plt.ylabel("Temperature")
plt.legend([
    "TEMP1", "TEMP2", "TEMP3",
    "TEMP4", "TEMP5"
])
plt.show()
```



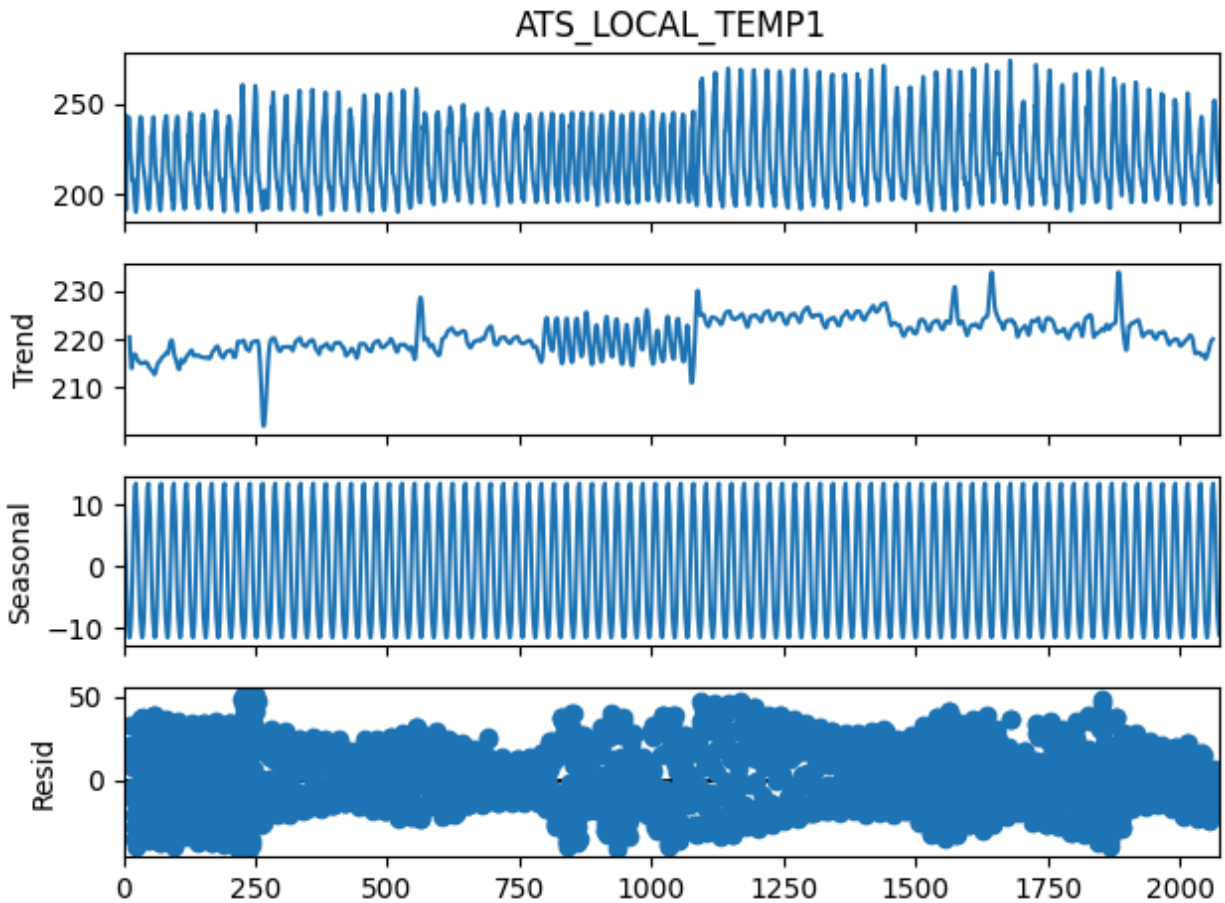
In [ ]:
```
# Average temperature per hour of day
df['Hour'] = df.index % 24

daily_avg = df.groupby('Hour')['ATS_LOCAL_TEMP1'].mean()

plt.figure()
plt.plot(daily_avg.values)
plt.title("Average Daily Temperature Pattern (24-Hour Cycle)")
plt.xlabel("Hour of Day")
plt.ylabel("Average Temperature")
plt.show()
```



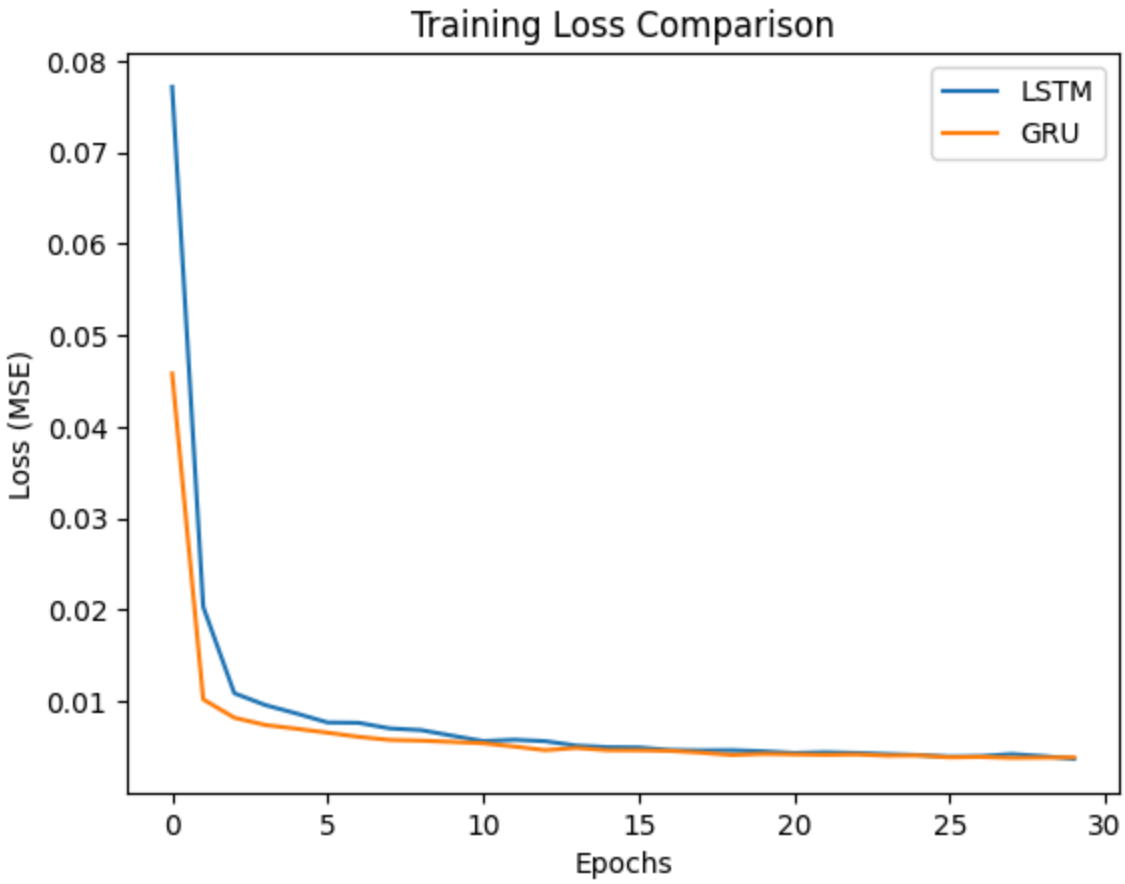
In [ ]:
```
rolling_mean = data['ATS_LOCAL_TEMP1'].rolling(window=168).mean()  # 168 hours

plt.figure()
plt.plot(data['ATS_LOCAL_TEMP1'][:1000])
plt.plot(rolling_mean[:1000])
plt.title("Trend Analysis using Rolling Mean")
plt.xlabel("Time")
plt.ylabel("Temperature")
plt.legend(["Original", "Weekly Trend"])
plt.show()
```



In [ ]:
```
from statsmodels.tsa.seasonal import seasonal_decompose

result = seasonal_decompose(
    data['ATS_LOCAL_TEMP1'],
    model='additive',
    period=24
)

result.plot()
plt.show()
```



In [ ]:
```
plt.figure()
plt.plot(history_lstm.history['loss'])
plt.plot(history_gru.history['loss'])
plt.title("Training Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend(["LSTM", "GRU"])
plt.show()
```


Training Loss Comparison
LSTM
GRU
Loss (MSE)
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0
5
10
15
20
25
30
Epochs